

Siniša Komaromi

Senior Game Developer

Location

Croatia

E-mail

sinisa.komaromi@gmail.com

Web

skomaromi.github.io

GitHub

github.com/skomaromi

01 SKILLS

Excellent proficiency with the following tools and software stacks:

Unity, C#, Git, GitLab CE, Linux.

Extensive experience with frameworks as well as programming and scripting languages, such as:

Unreal Engine, Python, C, C++, PowerShell, JavaScript, HTML, CSS, PHP, Bash.

Other technologies:

Photon Bolt, MLAPI, Zenject, Svelto ECS, PyGame, Qt, Inkscape, jQuery, D3.js.

02 EXPERIENCE

Senior Unity Developer – ARPG Project (Rock Paper Samurai, Binx Games)

OCT 2023 – PRESENT

- Built a highly modular ability framework driven by predefined lifecycle events and attribute values, usable by both player and enemy characters; built new abilities leveraging the framework per initial specifications and subsequent refinements; developed aiming strategies and accompanying indicator logic
- Implemented a player inventory system with items that can modify character stats and attribute values on abilities cast by the character
- Developed a shader-driven fog system which dynamically changes shape in response to changes in gameplay state and objects in the environment
- Introduced different enemy spawning modes and sources, overhauled game's enemy spawning system so that it's more modular and allows easier authoring and design iteration
- Implemented various quality-of-life features to facilitate faster iteration on gameplay experience like in-game cheats and editor tools

Senior Game Developer – Outfit7

FEB 2024 – JAN 2026

- Developed significant portions of game UI using Unity's uGUI system whilst taking care of different display layouts and cutouts as well as memory and draw call efficiency using data virtualization; collaborated with team members, leads and other departments to refine look and feel per feedback
- Wrote bespoke shader (Cg), runtime and editor code leveraging look-up textures to achieve palette-driven recoloring of in-game objects and characters; worked on addressing vendor-specific shader quirks
- Architected and implemented the core infrastructure defining character appearance and its on-disk representation when saving; made use of palette-driven recoloring functionality to allow configurable character and hairstyle colors; expanded infrastructure to accommodate for features introduced later in development cycle and, as the domain owner, provided guidance to other team members to aid its sustainable expansion

- Made extensive use of and implemented new building blocks using Unity's Timeline package to simplify the process of choreographing animation, VFX and SFX
- Implemented a bridge which leverages a third-party asset to discretize and refine parsed SVG geometry with the intention of eliminating long triangles and reducing geometry artifacts on rigged characters

Senior Unity Developer – Dapper Penguin Studios

APR 2023 – SEP 2023

- Implemented new gameplay elements and added features to existing ones, prioritizing minimal performance impact, within a project which utilizes ECS (Svelto ECS) in conjunction with Zenject, a DI framework
- Devised and developed a substantial portion of game's visual flow using Unity's UI Toolkit, integrating and adapting Figma art to meet requirements on UX and data to be displayed
- Contributed to the development of features which make use of HLSL shader code and the SRP API, simultaneously implementing counterparts in Unity's Shader Graph for comparative analysis when necessitated by issues
- Addressed technical issues and introduced new features using Unity's VFX Graph
- Worked on resolving reported code issues and revised features in response to feedback and evolving design requirements

Senior Unity Developer – Binx Games

JUL 2018 – PRESENT

- Implemented various gameplay features and iterated upon them based on feedback and design updates, for example:
 - a dependency tree which defined prerequisites before a certain building or item was unlocked - implemented both the rule definition code, usable by game designers, as well as gameplay experience and UI code;
 - a mechanism which, based on player's proximity to a certain in-game region, crossfaded visual effect contribution and defined which types of enemies were spawned
 - a modular system for defining actions upon enemy death
- Worked on gameplay features in an Unreal Engine project, such as:
 - implementing altering gameplay mechanics, adjusting world appearance by manipulating material properties at runtime, and updating the user interface - all in response to an in-game trigger or player input;
 - an enemy inventory system, which allows random weapon selection and weapon unequipping based on changes in world state;
 - creating new and modifying existing enemy behavior trees;
 - integrating visual effects according to design requirements
- Worked on implementing world streaming features, which included:
 - using Unity Addressable Asset System API to load and unload game content;
 - using features of the networking solution to toggle networked objects;
 - splitting content scenes in cells, processing terrain objects to generate low polygon count equivalents, working with third-party assets (MicroSplat) to ensure matching appearance between the stand-in object and the original one as well as using

network messaging to ensure both client and server have matching cell scenes loaded

- Established rich CI/CD integration, which included:
 - Unity code which allows defining, customizing and building game presets from the editor;
 - GitLab builder scripting with support for building any of previously defined presets and their deployment to target Steam branch with zero user intervention
- Implemented modular procedural level generation using a simplified two-dimensional random walk algorithm with corresponding editor UI to make preliminary result visualization easier
- Implemented a custom network messaging system for interactable in-game vegetation to reduce network overhead incurred by synchronizing scene vegetation as individual network objects
- Reworked existing world persistence solution to improve performance, usability and store more gameplay parts to the disk
- Implemented a world persistence solution from scratch designed with world streaming in mind with long-running operations (e.g. I/O tasks) spread over multiple frames; later integrated with previously existing game code
- Worked on porting the game from one networking solution to another (MLAPI to Photon Bolt), introducing server authority and client-side prediction for some gameplay features as part of the process
- Adapted third-party character controller solution (Kinematic Character Controller) for a server-authoritative networking library (Photon Bolt) while taking care of previously implemented movement-reliant gameplay code which relied on it
- Implemented support for multiple game store platforms (primarily Steam and Steamworks) to allow for client- and server-side ownership validation

03 EDUCATION

Master of Computer Engineering – University of Osijek, Croatia

OCT 2017 – SEP 2019

Developed various applications and games as part of course assignments, some of them leveraging network for its functionality, including:

- a 2D side-scrolling game with procedural level generation and voice control (Unity, C#)
- a simple turn-based rogue-like game (Python, PyGame)
- a real-time chat application with client and server counterparts with decentralized media storage (Python, WebSocket, Java, Django, REST, IPFS)
- an application which processes and displays satellite imagery in the form of a web map (Python, GDAL, Rasterio, Django)
- a single-page responsive web application for posting classifieds (Python, Django, REST)
- an image viewer extensible with user-created filters, was used as a part of a university image processing course (Python, Qt, OpenCV)
- an interactive map of Croatia displaying changes of indicators over time (HTML, CSS, D3.js)